

Simple Command Sequencer Generator

Autocoder User Guide

1 Description

The purpose of this generator is to provide a user-friendly way of defining named command sequences for the Simple Command Sequencer component. The generator takes a YAML model file as input which declares a suite of sequences – each a named, ordered list of sub-commands (and optional sleeps) – and autocodes four things from it: the Ada sequence step tables, bundled with the suite’s frame-pool size into a `Config` constant consumed by the sequencer’s `Init` routine; a set of first-class per-sequence wrapper commands so that each sequence appears on the sequencer instance as its own typed, operator-callable command; an operator-side command builder package for those wrapper commands; and the record type of the instance’s periodic summary packet.

Note the example shown in this documentation is used in the unit test of this component so that the reader of this document can see it being used in context. Please refer to the unit test code for more details on how this generator can be used.

2 Schema

The following pykwalify schema is used to validate the input YAML model. Model files must be named in the form *optional_name.assembly_name.command_sequences.yaml*, where *optional_name* is the specific name of this set of sequences (only needed if more than one suite is defined in the assembly) and *assembly_name* is the assembly the sequences will be used in. Generally this file is created in the same directory or near to the assembly model file. The schema is commented to show each available YAML key and what it accomplishes; even without knowing the specifics of pykwalify schemas, you should be able to glean the available structure from the file below.

```
1  # Schema for command_sequences YAML files
2  # This defines the structure and validation rules
3
4  type: map
5  mapping:
6    name:
7      required: false
8      type: str
9      desc: |
10         Optional human-readable name for this command sequences suite. The
11         generated Ada package name is NOT taken from here -- it is derived
12         from the model filename
13         (<model_name>_Command_Sequences[_<specific_name>]).
14
15    description:
16      required: false
17      type: str
18      desc: |
19         Description of this command sequences package. This will appear in
20         the generated Ada code comments.
21
22    preamble:
```

```

23     required: false
24     type: str
25     desc: |
26         Custom preamble text to include in the generated Ada package.
27
28 num_concurrent_sequences:
29     required: true
30     type: int
31     range: { min: 1 }
32     desc: |
33         The number of sequence frames (concurrently running sequences) a
34         sequencer instance initialized with this suite allocates. This also
35         sizes the suite's generated summary packet ground type (one
36         Sequence_Frame_Summary per frame), so the frame pool and the packet
37         layout can never disagree. The upper bound -- how many frame
38         summaries fit in one packet buffer -- is enforced by the model
39         against the project configuration.
40
41 command_timeout_seconds:
42     required: false
43     type: int
44     range: { min: 1, max: 2147483 }
45     desc: |
46         Default timeout, in whole seconds, that the sequencer will wait for
47         a sub-command's response before declaring the sequence failed. Used
48         for any sequence in this suite that does not specify its own
49         command_timeout_seconds. Bounded so the millisecond count always
50         fits an Ada.Real_Time.Time_Span.
51
52 with:
53     required: false
54     type: seq
55     desc: |
56         List of additional Ada packages to "with" in the generated sequences
57         package. Use this when a step's arg expression references a package
58         that is not pulled in automatically, e.g. an enum package for a
59         qualified enumeration literal.
60     sequence:
61         - type: str
62
63 sequences:
64     required: true
65     type: seq
66     desc: |
67         List of command sequence definitions. Each sequence defines a named
68         command that executes a series of sub-commands.
69     sequence:
70         - type: map
71           mapping:
72             name:
73                 required: true
74                 type: str
75                 desc: |
76                     Name of this sequence. This will become a command that can be
77                     executed. Must follow Ada naming conventions.
78
79             description:
80                 required: false
81                 type: str
82                 desc: |
83                     Description of what this sequence does. Will appear in generated

```

```

84         code and documentation.
85
86     arg_type:
87         required: false
88         type: str
89         desc: |
90             The Ada type for this sequence's argument. If not specified,
91             the sequence takes no arguments. Must be a valid Ada type name,
92             possibly qualified (e.g., "Power_Mode.T" or "Packed_U16.T").
93
94     wait_for_command_completion:
95         required: false
96         type: bool
97         default: true
98         desc: |
99             Default setting for whether to wait for each sub-command to
100             complete before executing the next one. Individual steps can
101             override this.
102
103     continue_on_failure:
104         required: false
105         type: bool
106         default: false
107         desc: |
108             If true, the sequence will continue executing even if a
109             ↪ sub-command
110             fails. If false, the sequence will abort on the first failure.
111
112     response_behavior:
113         required: false
114         type: str
115         enum: ['send_after_sequence_start',
116             ↪ 'send_after_sequence_completion']
117         desc: |
118             When the sequencer replies to this sequence's own command:
119             send_after_sequence_start (the default) acknowledges as soon as
120             the sequence is started; send_after_sequence_completion defers
121             the reply until the sequence finishes, carrying its final
122             success or failure. Static per-sequence configuration; the
123             generic Run_Sequence command uses it too.
124
125     command_timeout_seconds:
126         required: false
127         type: int
128         range: { min: 0, max: 2147483 }
129         desc: |
130             Override the suite-level default timeout for sub-command
131             responses, in whole seconds, for this sequence only. Bounded
132             so the millisecond count always fits an
133             Ada.Real_Time.Time_Span.
134
135     sequence:
136         required: true
137         type: seq
138         desc: |
139             The ordered list of steps to execute. Each step is either a
140             command dispatch (`command:` + optional `arg:`) or an in-sequence
141             sleep (`sleep_ms:` + integer milliseconds). The two forms are
142             mutually exclusive; exactly one must be present per step.
143             Validation is enforced in the model loader.
144     sequence:

```

```

143     - type: map
144     mapping:
145       command:
146         required: false
147         type: str
148         desc: |
149           The command to execute in the format
150           ↪ "Component.Command_Name".
151           The component must be a valid component instance name in
152           ↪ your
153           assembly, and the command must exist on that component.
154           Mutually exclusive with `sleep:`.
155
156       arg:
157         required: false
158         type: str
159         desc: |
160           An Ada expression that evaluates to the argument for this
161           command. You can reference the parent sequence's argument
162           using "Arg.field_name" syntax. If the command takes no
163           argument, omit this field. Only valid alongside
164           ↪ `command:`.
165
166       Examples:
167         - "Arg" - pass through the entire sequence argument
168         - "Arg.Power_Mode" - pass a field from the sequence
169           ↪ argument
170         - "(Mode => Arg.Mode, Enable => True)" - construct a
171           ↪ record
172         - "42" - a literal value
173
174     wait_for_completion:
175       required: false
176       type: bool
177       desc: |
178         Whether to wait for this specific command's response
179         ↪ before
180         proceeding. If not specified, uses the parent sequence's
181         wait_for_command_completion setting. Only valid alongside
182         `command:`.
183
184         A non-waiting step is fire-and-forget: its response is
185         discarded when it arrives, so a failed sub-command is
186         invisible (continue_on_failure does not apply) and the
187         step is never timed out. A non-waiting step followed later
188         in the same sequence by a waiting step on the same command
189         is rejected at generation time, since the stale response
190         could wake the waiting step early.
191
192     sleep_ms:
193       required: false
194       type: text
195       desc: |
196         Milliseconds to pause the sequence. Not bus-routed; the
197         sequencer frame parks itself for the duration without
198         issuing any command. Mutually exclusive with
199         `command:`/`arg:`/`wait_for_completion:`.
200
201         Either a static integer (0 .. 2**31 - 1, so it always fits
202         an Ada.Real_Time.Time_Span; enforced by the model), or a
203         dynamic reference into the sequence's argument using the

```

```

198         same "Arg[.field]..." syntax as dynamic command args. A
199         dynamic sleep's referenced field must be a
200         Packed_Natural.T, which the resolver validates at
201         execution time.
202
203         Examples:
204         - sleep_ms: 3000                # sleep 3 seconds
205         - sleep_ms: Arg.Settle_Time # duration from the sequence
206           ↪ argument
207
208         # Additional validation rules (enforced in the model Python code):
209         # - Command format must be "Component.Command" with valid identifiers, or
210         #   the reserved name "Sleep"
211         # - Referenced types in arg_type must exist in the system
212         # - Argument expressions must be valid Ada syntax
213         # - Component and command names must match instances in the assembly

```

3 Example Input

The following is an example command sequences input YAML file. It defines twelve sequences exercising the main features: statically-armed commands and static sleeps, typed argument records whose fields are dispatched as dynamic arguments to sub-commands (including through non-byte-aligned packed fields and validatable records), a dynamic sleep whose duration comes from the sequence argument, no-wait direct dispatch (sequence-wide and per-step via `wait_for_completion`), a sequence that calls another sequence as a sub-sequence step, and sequences whose command response is statically deferred until completion. This example adheres to the schema shown in the previous section, and is commented to give clarification.

Note the optional top-level `with:` key, which lists additional Ada packages to “with” in the generated package. A step’s `arg:` expression is emitted verbatim into the generated Ada, so any package it references that is not pulled in automatically – such as an enum package needed to qualify an enumeration literal – must be listed here.

```

1  # Command Sequences Definition
2  # This file defines composite commands that execute sequences of sub-commands
3
4  description: Example command sequences exercising the generator's features
5
6  # Two frames: two sequences may run concurrently, and the generated summary
7  # packet type carries two frame summaries.
8  num_concurrent_sequences: 2
9
10 command_timeout_seconds: 10 # default wait for sub-command responses
11
12 sequences:
13   # Simple sequence with no arguments
14   - name: Sequence_A
15     description: Execute commands on component A
16     wait_for_command_completion: true
17     continue_on_failure: true
18
19   sequence:
20     - command: Component_A.Command_1
21     - command: Component_A.Command_2
22       arg: (Seconds => 3, Subseconds => 14)
23     - sleep_ms: 3000
24     - command: Component_A.Command_3
25       arg: (Value => 3)

```

```

26
27 - name: Sequence_B
28   description: Perform generic arguments with component A and B
29   wait_for_command_completion: true
30   continue_on_failure: false
31   arg_type: Sequence_B_Arg.T
32
33   sequence:
34     - command: Component_A.Command_3
35       arg: Arg.Component_A_Arg
36     - command: Component_B.Command_3
37       arg: Arg.Component_B_Arg
38
39 - name: Sequence_C
40   description: No-wait sequence for direct-dispatch testing
41   wait_for_command_completion: false
42   continue_on_failure: false
43
44   sequence:
45     - command: Component_A.Command_1
46     - command: Component_A.Command_1
47
48 - name: Sequence_D
49   description: Sequence with a dynamic sleep duration taken from the sequence
50     ↪ argument
51   wait_for_command_completion: true
52   continue_on_failure: false
53   arg_type: Packed_Natural.T
54
55   sequence:
56     - sleep_ms: Arg
57
58   # Calls another sequence. Each sequence is a command on the sequencer, so a
59   # sub-sequence step is a command step targeting the sequencer's own command.
60   # Sequence_C replies on start, so this sequence resumes as soon as it starts.
61 - name: Sequence_E
62   description: Call Sequence_C as a sub-sequence, then run a regular command
63   wait_for_command_completion: true
64   continue_on_failure: false
65
66   sequence:
67     - command: Sequencer.Sequence_C
68     - command: Component_A.Command_1
69
70   # Command response deferred until completion: the operator's reply carries
71   # the final success/failure instead of a start acknowledgement.
72 - name: Sequence_F
73   description: No-wait sequence with a statically deferred command response
74   wait_for_command_completion: false
75   continue_on_failure: false
76   response_behavior: send_after_sequence_completion
77
78   sequence:
79     - command: Component_A.Command_1
80
81   # Deferred response and waits on its sub-command, so the reply arrives when
82   # the sub-command completes (or the sequence aborts, times out, or is killed).
83 - name: Sequence_G
84   description: Waiting sequence with a statically deferred command response
85   wait_for_command_completion: true
86   continue_on_failure: false

```

```

86     response_behavior: send_after_sequence_completion
87
88     sequence:
89         - command: Component_A.Command_1
90
91     # Dynamic-arg repacking: the argument's leading 4-bit pad puts both leaf
92     # fields at non-byte-aligned offsets, so byte-offset slicing would corrupt
93     ↳ them.
94     - name: Sequence_H
95       description: Dynamic steps traversing misaligned 8-bit and 16-bit packed
96       ↳ fields
97       wait_for_command_completion: true
98       continue_on_failure: false
99       arg_type: Nibble_Test_Args.T
100
101     sequence:
102         - command: Component_A.Command_8
103           arg: Arg.Wide
104         - command: Component_A.Command_7
105           arg: Arg.Nibble
106
107     # First step is a static sleep, so wake-time overflow can be exercised
108     # without a command step's response deadline overflowing first.
109     - name: Sequence_I
110       description: A lone static sleep, for wake-time overflow coverage
111       wait_for_command_completion: true
112       continue_on_failure: false
113
114     sequence:
115         - sleep_ms: 3000
116
117     # The two sequences below take Validated_Args, whose Mode enumeration can
118     # fail validation, for the invalid-dynamic-argument failure paths.
119     - name: Sequence_J
120       description: Dynamic command argument resolved from a validatable record
121       wait_for_command_completion: true
122       continue_on_failure: false
123       arg_type: Validated_Args.T
124
125     sequence:
126         - command: Component_A.Command_3
127           arg: Arg.Duration
128
129     - name: Sequence_K
130       description: Dynamic sleep resolved from a validatable record
131       wait_for_command_completion: true
132       continue_on_failure: false
133       arg_type: Validated_Args.T
134
135     sequence:
136         - sleep_ms: Arg.Sleep_Duration
137
138     # Mixed per-step waits: the first command dispatches fire-and-forget (its
139     # response is discarded, so a failure is invisible), and the second is
140     # confirmed before the sequence completes.
141     - name: Sequence_L
142       description: A no-wait config command followed by a confirmed command
143       wait_for_command_completion: true
144       continue_on_failure: false
145
146     sequence:

```

```

145     - command: Component_A.Command_1
146       wait_for_completion: false
147     - command: Component_A.Command_3
148       arg: (Value => 7)

```

4 Example Output

The example input above produces the following Ada outputs. The package spec declares the To_Arg buffer-padding helper, the per-dynamic-step Resolver functions, a step array constant per sequence, the Sequences_Table constant (plus a stable Sequences access pointer), and the suite's Config constant, which bundles the table access with the suite's num_concurrent_sequences. The package body implements the Resolver functions: each validates the sequence's per-call argument buffer through the input type's generated Validation.Valid, deserializes it through Serialization.From_Byte_Array, and re-serializes the addressed field as the step's argument buffer (a sub-command's argument, or a Packed_Natural millisecond count for a dynamic sleep). Resolvers exist only for dynamic steps – a suite of purely static sequences declares none, and its body degenerates to pragma No_Body;.

The Config constant should be passed into the Simple Command Sequencer's Init procedure during assembly initialization, e.g. in the assembly YAML:

```

- type: Simple_Command_Sequencer
  name: My_Sequencer
  init:
    - "Config => Test_Assembly_Command_Sequences_Example_Sequences.Config"

```

```

1  -- Auto-generated by Adamant command sequences generator.
2  -- Do not edit by hand.
3  -- Example command sequences exercising the generator's features
4  with Simple_Sequencer_Types; use Simple_Sequencer_Types;
5  with Test_Assembly_Commands; use Test_Assembly_Commands;
6  with Command_Types;
7  with Basic_Types;
8  with Sequence_Enums;
9  with Ada.Real_Time;
10 with Sys_Time_32;
11 with Packed_U32;
12 with Sequence_B_Arg;
13 with Packed_Natural;
14 with Nibble_Test_Args;
15 with Packed_U16;
16 with Packed_Nibble;
17 with Validated_Args;
18 package Test_Assembly_Command_Sequences_Example_Sequences is
19   -- Make the "&" operator on Byte_Array visible for To_Arg below.
20   use type Basic_Types.Byte_Array;
21
22   -----
23   -- Pad a serialized argument out to a full command argument buffer. An
24   -- expression function so the step arrays below can call it at elaboration.
25   -----
26   function To_Arg (Bytes : Basic_Types.Byte_Array) return
27     ↪ Command_Types.Command_Arg_Buffer_Type is
28     (Command_Types.Command_Arg_Buffer_Type (Bytes & Basic_Types.Byte_Array'(1
29       ↪ .. Command_Types.Command_Arg_Buffer_Type'Length - Bytes'Length =>
30       ↪ 0)));
31
32   -----
33   -- Resolvers - one per dynamic step, each extracting one field of the

```

```

30  -- sequence argument.
31  -----
32  function Sequence_B_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
33  ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean;
34  function Sequence_B_Step_1_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
35  ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean;
36  function Sequence_D_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
37  ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean;
38  function Sequence_H_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
39  ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean;
40  function Sequence_H_Step_1_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
41  ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean;
42  function Sequence_J_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
43  ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean;
44  function Sequence_K_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
45  ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean;
46  -----
47  -- Step arrays - one constant per sequence.
48  -----
49  Sequence_A_Steps : aliased constant Step_Array :=
50  [
51    0 =>
52      (Kind      => Command_Step,
53       Id        => Component_A_Command_1,
54       Arg_Length => 0,
55       Wait_For_Cmd_Resp => True,
56       Arg        => [others => 0]),
57    1 =>
58      (Kind      => Command_Step,
59       Id        => Component_A_Command_2,
60       Arg_Length => Sys_Time_32.Serialization.Serialized_Length,
61       Wait_For_Cmd_Resp => True,
62       Arg        => To_Arg (Sys_Time_32.Serialization.To_Byte_Array
63         ↪ ((Seconds => 3, Subseconds => 14)))),
64    2 =>
65      (Kind      => Sleep,
66       Id        => 0,
67       Arg_Length => 0,
68       Wait_For_Cmd_Resp => False,
69       Sleep_Arg  => 3000),
70    3 =>
71      (Kind      => Command_Step,
72       Id        => Component_A_Command_3,
73       Arg_Length => Packed_U32.Serialization.Serialized_Length,
74       Wait_For_Cmd_Resp => True,
75       Arg        => To_Arg (Packed_U32.Serialization.To_Byte_Array ((Value
76         ↪ => 3))))
77  ];
78  Sequence_B_Steps : aliased constant Step_Array :=
79  [
80    0 =>
81      (Kind      => Runtime_Argument_Command_Step,
82       Id        => Component_A_Command_3,
83       Arg_Length => Packed_U32.Serialization.Serialized_Length,
84       Wait_For_Cmd_Resp => True,
85       Resolver   => Sequence_B_Step_0_Resolver_T'Access),
86    1 =>
87      (Kind      => Runtime_Argument_Command_Step,
88       Id        => Component_B_Command_3,
89       Arg_Length => Packed_U32.Serialization.Serialized_Length,
90       Wait_For_Cmd_Resp => True,

```

```

82         Resolver    => Sequence_B_Step_1_Resolver_T'Access)
83     ];
84     Sequence_C_Steps : aliased constant Step_Array :=
85     [
86         0 =>
87             (Kind      => Command_Step,
88              Id        => Component_A_Command_1,
89              Arg_Length => 0,
90              Wait_For_Cmd_Resp => False,
91              Arg       => [others => 0]),
92         1 =>
93             (Kind      => Command_Step,
94              Id        => Component_A_Command_1,
95              Arg_Length => 0,
96              Wait_For_Cmd_Resp => False,
97              Arg       => [others => 0])
98     ];
99     Sequence_D_Steps : aliased constant Step_Array :=
100    [
101        0 =>
102            (Kind      => Runtime_Sleep,
103             Id        => 0,
104             Arg_Length => Packed_Natural.Serialization.Serialized_Length,
105             Wait_For_Cmd_Resp => False,
106             Sleep_Resolver => Sequence_D_Step_0_Resolver_T'Access)
107    ];
108     Sequence_E_Steps : aliased constant Step_Array :=
109    [
110        0 =>
111            (Kind      => Command_Step,
112             Id        => Sequencer_Sequence_C,
113             Arg_Length => 0,
114             Wait_For_Cmd_Resp => True,
115             Arg       => [others => 0]),
116        1 =>
117            (Kind      => Command_Step,
118             Id        => Component_A_Command_1,
119             Arg_Length => 0,
120             Wait_For_Cmd_Resp => True,
121             Arg       => [others => 0])
122    ];
123     Sequence_F_Steps : aliased constant Step_Array :=
124    [
125        0 =>
126            (Kind      => Command_Step,
127             Id        => Component_A_Command_1,
128             Arg_Length => 0,
129             Wait_For_Cmd_Resp => False,
130             Arg       => [others => 0])
131    ];
132     Sequence_G_Steps : aliased constant Step_Array :=
133    [
134        0 =>
135            (Kind      => Command_Step,
136             Id        => Component_A_Command_1,
137             Arg_Length => 0,
138             Wait_For_Cmd_Resp => True,
139             Arg       => [others => 0])
140    ];
141     Sequence_H_Steps : aliased constant Step_Array :=
142    [

```

```

143     0 =>
144         (Kind      => Runtime_Argument_Command_Step,
145          Id        => Component_A_Command_8,
146          Arg_Length => Packed_U16.Serialization.Serialized_Length,
147          Wait_For_Cmd_Resp => True,
148          Resolver   => Sequence_H_Step_0_Resolver_T'Access),
149     1 =>
150         (Kind      => Runtime_Argument_Command_Step,
151          Id        => Component_A_Command_7,
152          Arg_Length => Packed_Nibble.Serialization.Serialized_Length,
153          Wait_For_Cmd_Resp => True,
154          Resolver   => Sequence_H_Step_1_Resolver_T'Access)
155 ];
156 Sequence_I_Steps : aliased constant Step_Array :=
157 [
158     0 =>
159         (Kind      => Sleep,
160          Id        => 0,
161          Arg_Length => 0,
162          Wait_For_Cmd_Resp => False,
163          Sleep_Arg  => 3000)
164 ];
165 Sequence_J_Steps : aliased constant Step_Array :=
166 [
167     0 =>
168         (Kind      => Runtime_Argument_Command_Step,
169          Id        => Component_A_Command_3,
170          Arg_Length => Packed_U32.Serialization.Serialized_Length,
171          Wait_For_Cmd_Resp => True,
172          Resolver   => Sequence_J_Step_0_Resolver_T'Access)
173 ];
174 Sequence_K_Steps : aliased constant Step_Array :=
175 [
176     0 =>
177         (Kind      => Runtime_Sleep,
178          Id        => 0,
179          Arg_Length => Packed_Natural.Serialization.Serialized_Length,
180          Wait_For_Cmd_Resp => False,
181          Sleep_Resolver => Sequence_K_Step_0_Resolver_T'Access)
182 ];
183 Sequence_L_Steps : aliased constant Step_Array :=
184 [
185     0 =>
186         (Kind      => Command_Step,
187          Id        => Component_A_Command_1,
188          Arg_Length => 0,
189          Wait_For_Cmd_Resp => False,
190          Arg        => [others => 0]),
191     1 =>
192         (Kind      => Command_Step,
193          Id        => Component_A_Command_3,
194          Arg_Length => Packed_U32.Serialization.Serialized_Length,
195          Wait_For_Cmd_Resp => True,
196          Arg        => To_Arg (Packed_U32.Serialization.To_Byte_Array ((Value
197                               ↪ => 7)))))
197 ];
198 -----
199 -- Sequences_Table
200 -----
201 Sequences_Table : aliased constant Sequences_Type :=
202 [

```

```

203     0 =>
204         (Abort_On_Failed_Cmd => False,
205          Command_Timeout     => Ada.Real_Time.Milliseconds (10000),
206          Response_Behavior   => Sequence_Enums.Sequence_Response_Behavior.Send,
207          ↪ _After_Sequence_Start,
208          Arg_Length          => 0,
209          Steps                => Sequence_A_Steps'Access),
210     1 =>
211         (Abort_On_Failed_Cmd => True,
212          Command_Timeout     => Ada.Real_Time.Milliseconds (10000),
213          Response_Behavior   => Sequence_Enums.Sequence_Response_Behavior.Send,
214          ↪ _After_Sequence_Start,
215          Arg_Length          => Sequence_B_Arg.Serialization.Serialized_Length,
216          Steps                => Sequence_B_Steps'Access),
217     2 =>
218         (Abort_On_Failed_Cmd => True,
219          Command_Timeout     => Ada.Real_Time.Milliseconds (10000),
220          Response_Behavior   => Sequence_Enums.Sequence_Response_Behavior.Send,
221          ↪ _After_Sequence_Start,
222          Arg_Length          => 0,
223          Steps                => Sequence_C_Steps'Access),
224     3 =>
225         (Abort_On_Failed_Cmd => True,
226          Command_Timeout     => Ada.Real_Time.Milliseconds (10000),
227          Response_Behavior   => Sequence_Enums.Sequence_Response_Behavior.Send,
228          ↪ _After_Sequence_Start,
229          Arg_Length          => Packed_Natural.Serialization.Serialized_Length,
230          Steps                => Sequence_D_Steps'Access),
231     4 =>
232         (Abort_On_Failed_Cmd => True,
233          Command_Timeout     => Ada.Real_Time.Milliseconds (10000),
234          Response_Behavior   => Sequence_Enums.Sequence_Response_Behavior.Send,
235          ↪ _After_Sequence_Start,
236          Arg_Length          => 0,
237          Steps                => Sequence_E_Steps'Access),
238     5 =>
239         (Abort_On_Failed_Cmd => True,
240          Command_Timeout     => Ada.Real_Time.Milliseconds (10000),
241          Response_Behavior   => Sequence_Enums.Sequence_Response_Behavior.Send,
242          ↪ _After_Sequence_Completion,
243          Arg_Length          => 0,
244          Steps                => Sequence_F_Steps'Access),
245     6 =>
246         (Abort_On_Failed_Cmd => True,
247          Command_Timeout     => Ada.Real_Time.Milliseconds (10000),
248          Response_Behavior   => Sequence_Enums.Sequence_Response_Behavior.Send,
249          ↪ _After_Sequence_Completion,
250          Arg_Length          => 0,
251          Steps                => Sequence_G_Steps'Access),
252     7 =>
253         (Abort_On_Failed_Cmd => True,
254          Command_Timeout     => Ada.Real_Time.Milliseconds (10000),
255          Response_Behavior   => Sequence_Enums.Sequence_Response_Behavior.Send,
256          ↪ _After_Sequence_Start,
257          Arg_Length          =>
258              ↪ Nibble_Test_Args.Serialization.Serialized_Length,
259          Steps                => Sequence_H_Steps'Access),
260     8 =>
261         (Abort_On_Failed_Cmd => True,
262          Command_Timeout     => Ada.Real_Time.Milliseconds (10000),
263          Response_Behavior   => Sequence_Enums.Sequence_Response_Behavior.Send,
264          ↪ _After_Sequence_Start,

```

```

255     Arg_Length      => 0,
256     Steps           => Sequence_I_Steps'Access),
257 9 =>
258     (Abort_On_Failed_Cmd => True,
259     Command_Timeout    => Ada.Real_Time.Milliseconds (10000),
260     Response_Behavior  => Sequence_Enums.Sequence_Response_Behavior.Send_
    ↪ _After_Sequence_Start,
261     Arg_Length        => Validated_Args.Serialization.Serialized_Length,
262     Steps             => Sequence_J_Steps'Access),
263 10 =>
264     (Abort_On_Failed_Cmd => True,
265     Command_Timeout    => Ada.Real_Time.Milliseconds (10000),
266     Response_Behavior  => Sequence_Enums.Sequence_Response_Behavior.Send_
    ↪ _After_Sequence_Start,
267     Arg_Length        => Validated_Args.Serialization.Serialized_Length,
268     Steps             => Sequence_K_Steps'Access),
269 11 =>
270     (Abort_On_Failed_Cmd => True,
271     Command_Timeout    => Ada.Real_Time.Milliseconds (10000),
272     Response_Behavior  => Sequence_Enums.Sequence_Response_Behavior.Send_
    ↪ _After_Sequence_Start,
273     Arg_Length        => 0,
274     Steps             => Sequence_L_Steps'Access)
275 ];
276
277 Sequences : constant Sequences_Access := Sequences_Table'Access;
278
279 -- Passed to Simple_Command_Sequencer.Init. num_concurrent_sequences also
280 -- sizes the generated
    ↪ Test_Assembly_Command_Sequences_Example_Sequences_Summary_Record packet
    ↪ type, so the frame
281 -- pool and the packet layout cannot disagree.
282 Config : constant Sequencer_Config :=
283     (Sequences => Sequences_Table'Access,
284     Num_Concurrent_Sequences => 2);
285 end Test_Assembly_Command_Sequences_Example_Sequences;

```

```

1  -- Auto-generated by Adamant command sequences generator.
2  -- Do not edit by hand.
3  with Interfaces;
4  with Sequence_B_Arg.Validation;
5  with Packed_Natural.Validation;
6  with Nibble_Test_Args.Validation;
7  with Validated_Args.Validation;
8  package body Test_Assembly_Command_Sequences_Example_Sequences is
9
10     -----
11     -- Resolvers - one per dynamic step. Each validates and deserializes the
12     -- sequence argument, then returns the addressed field serialized (a
13     -- sub-command argument, or a Packed_Natural millisecond count for a sleep).
14     -----
15     function Sequence_B_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
    ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean is
16     Errant_Field : Interfaces.Unsigned_32;
17     Input : Sequence_B_Arg.T;
18     Valid : constant Boolean := Sequence_B_Arg.Validation.Valid (Bytes
    ↪ (Bytes'First .. Bytes'First +
    ↪ Sequence_B_Arg.Serialization.Serialized_Length - 1), Errant_Field);
19 begin
20     if Valid then

```

```

21     Input := Sequence_B_Arg.Serialization.From_Byte_Array (Bytes
    ↪ (Bytes'First .. Bytes'First +
    ↪ Sequence_B_Arg.Serialization.Serialized_Length - 1));
22     Args := To_Arg (Packed_U32.Serialization.To_Byte_Array
    ↪ (Input.Component_A_Arg));
23     end if;
24     return Valid;
25 end Sequence_B_Step_0_Resolver_T;
26 function Sequence_B_Step_1_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
    ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean is
27     Errant_Field : Interfaces.Unsigned_32;
28     Input : Sequence_B_Arg.T;
29     Valid : constant Boolean := Sequence_B_Arg.Validation.Valid (Bytes
    ↪ (Bytes'First .. Bytes'First +
    ↪ Sequence_B_Arg.Serialization.Serialized_Length - 1), Errant_Field);
30 begin
31     if Valid then
32         Input := Sequence_B_Arg.Serialization.From_Byte_Array (Bytes
    ↪ (Bytes'First .. Bytes'First +
    ↪ Sequence_B_Arg.Serialization.Serialized_Length - 1));
33         Args := To_Arg (Packed_U32.Serialization.To_Byte_Array
    ↪ (Input.Component_B_Arg));
34     end if;
35     return Valid;
36 end Sequence_B_Step_1_Resolver_T;
37 function Sequence_D_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
    ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean is
38     Errant_Field : Interfaces.Unsigned_32;
39     Input : Packed_Natural.T;
40     Valid : constant Boolean := Packed_Natural.Validation.Valid (Bytes
    ↪ (Bytes'First .. Bytes'First +
    ↪ Packed_Natural.Serialization.Serialized_Length - 1), Errant_Field);
41 begin
42     if Valid then
43         Input := Packed_Natural.Serialization.From_Byte_Array (Bytes
    ↪ (Bytes'First .. Bytes'First +
    ↪ Packed_Natural.Serialization.Serialized_Length - 1));
44         Args := To_Arg (Packed_Natural.Serialization.To_Byte_Array (Input));
45     end if;
46     return Valid;
47 end Sequence_D_Step_0_Resolver_T;
48 function Sequence_H_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
    ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean is
49     Errant_Field : Interfaces.Unsigned_32;
50     Input : Nibble_Test_Args.T;
51     Valid : constant Boolean := Nibble_Test_Args.Validation.Valid (Bytes
    ↪ (Bytes'First .. Bytes'First +
    ↪ Nibble_Test_Args.Serialization.Serialized_Length - 1), Errant_Field);
52 begin
53     if Valid then
54         Input := Nibble_Test_Args.Serialization.From_Byte_Array (Bytes
    ↪ (Bytes'First .. Bytes'First +
    ↪ Nibble_Test_Args.Serialization.Serialized_Length - 1));
55         Args := To_Arg (Packed_U16.Serialization.To_Byte_Array (Input.Wide));
56     end if;
57     return Valid;
58 end Sequence_H_Step_0_Resolver_T;
59 function Sequence_H_Step_1_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
    ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean is
60     Errant_Field : Interfaces.Unsigned_32;
61     Input : Nibble_Test_Args.T;

```

```

62     Valid : constant Boolean := Nibble_Test_Args.Validation.Valid (Bytes
    ↪ (Bytes'First .. Bytes'First +
    ↪ Nibble_Test_Args.Serialization.Serialized_Length - 1), Errant_Field);
63 begin
64     if Valid then
65         Input := Nibble_Test_Args.Serialization.From_Byte_Array (Bytes
    ↪ (Bytes'First .. Bytes'First +
    ↪ Nibble_Test_Args.Serialization.Serialized_Length - 1));
66         Args := To_Arg (Packed_Nibble.Serialization.To_Byte_Array
    ↪ (Input.Nibble));
67     end if;
68     return Valid;
69 end Sequence_H_Step_1_Resolver_T;
70 function Sequence_J_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
    ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean is
71     Errant_Field : Interfaces.Unsigned_32;
72     Input : Validated_Args.T;
73     Valid : constant Boolean := Validated_Args.Validation.Valid (Bytes
    ↪ (Bytes'First .. Bytes'First +
    ↪ Validated_Args.Serialization.Serialized_Length - 1), Errant_Field);
74 begin
75     if Valid then
76         Input := Validated_Args.Serialization.From_Byte_Array (Bytes
    ↪ (Bytes'First .. Bytes'First +
    ↪ Validated_Args.Serialization.Serialized_Length - 1));
77         Args := To_Arg (Packed_U32.Serialization.To_Byte_Array
    ↪ (Input.Duration));
78     end if;
79     return Valid;
80 end Sequence_J_Step_0_Resolver_T;
81 function Sequence_K_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
    ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean is
82     Errant_Field : Interfaces.Unsigned_32;
83     Input : Validated_Args.T;
84     Valid : constant Boolean := Validated_Args.Validation.Valid (Bytes
    ↪ (Bytes'First .. Bytes'First +
    ↪ Validated_Args.Serialization.Serialized_Length - 1), Errant_Field);
85 begin
86     if Valid then
87         Input := Validated_Args.Serialization.From_Byte_Array (Bytes
    ↪ (Bytes'First .. Bytes'First +
    ↪ Validated_Args.Serialization.Serialized_Length - 1));
88         Args := To_Arg (Packed_Natural.Serialization.To_Byte_Array
    ↪ (Input.Sleep_Duration));
89     end if;
90     return Valid;
91 end Sequence_K_Step_0_Resolver_T;
92 end Test_Assembly_Command_Sequences_Example_Sequences;
93

```

5 Per-Sequence First-Class Commands

In addition to the sequence tables above, this same model drives the autocoded synthesis of one first-class command per declared sequence on the Simple Command Sequencer instance. Discovery is init-parameter-driven: at assembly build time the sequencer's commands model reads each instance's Config init parameter, derives the suite package named there (e.g. `Test_Assembly_Command_Sequences_Example_Sequences.Config`), and loads exactly that suite's model, injecting one command per declared sequence into the instance's command set. A naked component build (no assembly, hence no instance and no Config value) produces only the four static

commands (Run_Sequence, Kill_All-Sequences, Kill_Frame, Set_Summary_Packet_Period); a suite merely present on the build path contributes nothing – only the suite an instance names in its Config adds commands, so two sequencer instances in one assembly can carry different suites. Each per-sequence command (e.g. Sequence_B) takes the sequence’s own arg_type verbatim as its command argument (or no argument for an argless sequence) – a user-written, normally-registered type, so no argument records are generated or registered anywhere in the pipeline. At run time the component copies the received argument buffer unchanged into a generic Run_Sequence_Arg.T and dispatches through the existing Run_Sequence backbone using the sequence’s runtime slot index. Run_Sequence rejects any argument whose length differs from the serialized length of the sequence’s arg_type (recorded in the autocoded table) with an Invalid_Sequence_Argument_Length event; the response behavior comes from the sequence’s static configuration in the same table (see Sequence Response Behavior below). A companion generated package, <suite>_Commands, carries the operator-side builder surface (per-sequence command id getters and Command.T constructors) for unit tests and other on-board callers – split from the sequences package because the sequencer component itself never uses the builders. No per-sequence Ada needs to be hand-written:

```

1  -- Auto-generated by Adamant command sequences generator.
2  -- Do not edit by hand.
3  -- Example command sequences exercising the generator's features
4  --
5  -- Operator-side builders (id getters and Command.T constructors) for the
6  -- Test_Assembly_Command_Sequences_Example_Sequences suite's per-sequence
   ↪ commands, for unit tests and on-board
7  -- callers. The sequencer component itself does not use this package.
8  with Command;
9  with Command_Types;
10 with Nibble_Test_Args;
11 with Packed_Natural;
12 with Sequence_B_Arg;
13 with Validated_Args;
14 package Test_Assembly_Command_Sequences_Example_Sequences_Commands is
15
16     -- Set Id_Base to the sequencer instance's command id base. Each sequence
17     -- command id is Id_Base + Num_Commands + declaration index.
18     type Instance is tagged private;
19
20     -- Set the sequencer instance's command id base so constructed commands
   ↪ carry
21     -- ids that match what the component registered at runtime.
22     procedure Set_Id_Base (Self : in out Instance; Id_Base : in
   ↪ Command_Types.Command_Id);
23
24     -- Set the operator sourceid stamped into the header of constructed commands
25     -- (defaults to 0). Mirrors the command suite's Set_Source_Id.
26     procedure Set_Source_Id (Self : in out Instance; Source_Id : in
   ↪ Command_Types.Command_Source_Id);
27
28     function Get_Sequence_A_Id (Self : in Instance) return
   ↪ Command_Types.Command_Id;
29     function Sequence_A (Self : in Instance) return Command.T;
30     function Get_Sequence_B_Id (Self : in Instance) return
   ↪ Command_Types.Command_Id;
31     function Sequence_B (Self : in Instance; Arg : in Sequence_B_Arg.T) return
   ↪ Command.T;
32     function Get_Sequence_C_Id (Self : in Instance) return
   ↪ Command_Types.Command_Id;
33     function Sequence_C (Self : in Instance) return Command.T;
34     function Get_Sequence_D_Id (Self : in Instance) return
   ↪ Command_Types.Command_Id;

```

```

35     function Sequence_D (Self : in Instance; Arg : in Packed_Natural.T) return
        ↪ Command.T;
36     function Get_Sequence_E_Id (Self : in Instance) return
        ↪ Command_Types.Command_Id;
37     function Sequence_E (Self : in Instance) return Command.T;
38     function Get_Sequence_F_Id (Self : in Instance) return
        ↪ Command_Types.Command_Id;
39     function Sequence_F (Self : in Instance) return Command.T;
40     function Get_Sequence_G_Id (Self : in Instance) return
        ↪ Command_Types.Command_Id;
41     function Sequence_G (Self : in Instance) return Command.T;
42     function Get_Sequence_H_Id (Self : in Instance) return
        ↪ Command_Types.Command_Id;
43     function Sequence_H (Self : in Instance; Arg : in Nibble_Test_Args.T) return
        ↪ Command.T;
44     function Get_Sequence_I_Id (Self : in Instance) return
        ↪ Command_Types.Command_Id;
45     function Sequence_I (Self : in Instance) return Command.T;
46     function Get_Sequence_J_Id (Self : in Instance) return
        ↪ Command_Types.Command_Id;
47     function Sequence_J (Self : in Instance; Arg : in Validated_Args.T) return
        ↪ Command.T;
48     function Get_Sequence_K_Id (Self : in Instance) return
        ↪ Command_Types.Command_Id;
49     function Sequence_K (Self : in Instance; Arg : in Validated_Args.T) return
        ↪ Command.T;
50     function Get_Sequence_L_Id (Self : in Instance) return
        ↪ Command_Types.Command_Id;
51     function Sequence_L (Self : in Instance) return Command.T;
52
53 private
54     type Instance is tagged record
55         Id_Base : Command_Types.Command_Id := 0;
56         Source_Id : Command_Types.Command_Source_Id := 0;
57     end record;
58 end Test_Assembly_Command_Sequences_Example_Sequences_Commands;

```

6 Sequences Calling Sequences

Because every sequence is a first-class command on the sequencer instance, a sequence step may invoke another sequence simply by targeting the sequencer's own synthesized command, e.g. `command: My_Sequencer.Sequence_C`. No special syntax or generator support is involved – the step resolves like any other `Component.Command` reference, provided the sequencer instance is declared in the assembly. At run time the sub-sequence command is emitted out the sequencer's `Command_T_Send` connector, routed through the command router, and arrives back at the sequencer's `Command_T_Recv_Async`, where it claims its own frame and executes concurrently with (and independently of) the calling sequence. If the calling sequence waits for command completion, the sub-sequence's command response (per its static `response_behavior` configuration, see below) is what wakes it. Nesting depth is naturally capped by `Num_Concurrent_Sequences`: a sub-sequence call when no frame is available is rejected with a `No_Frame_Available` event and a `Failure` response to the calling frame.

7 Sequence Response Behavior

When the sequencer replies to the command that started a sequence is controlled by the `Sequence_Response_Behavior` enumeration. It is static per-sequence configuration: the optional `response_behavior` field on each sequence in the YAML, baked into the autocoded sequence table and used

by both the synthesized per-sequence commands and the generic `Run_Sequence` command:

- `Send_After_Sequence_Start` (the conventional default) – the command response is sent immediately once the sequence has been accepted and started on a frame. This is the right choice for fire-and-forget operation and for sub-sequence calls where the caller should proceed as soon as the sub-sequence is running.
- `Send_After_Sequence_Completion` – the command response is withheld until the sequence ends, and its status reflects the outcome: `Success` when the final step completes, or `Failure` when the sequence is aborted by a failed sub-command, times out waiting for a sub-command response, fails to resolve or validate a dynamic argument, encounters an out-of-range sleep or timeout, or is halted by `Kill_All-Sequences` or `Kill_Frame`. The sequencer captures the originating command’s source and id per frame, so concurrent deferred sequences each reply to their own operator.

A sequence step may forward a field of the parent sequence’s argument to the sub-command it dispatches, using the `Arg.field.subfield` syntax in the step’s `arg:`. Each such reference produces a typed Resolver in the generated package: at run time the sequencer dispatches that Resolver, which first validates the per-call argument buffer through the input type’s generated `Validation.Valid` (a failure aborts the sequence with an `Invalid_Dynamic_Command_Argument` event), then deserializes it through the input type’s safe `Serialization.From_Byte_Array`, navigates to the addressed field, and re-serializes it as the sub-command’s argument buffer. The resolver’s signature accepts only a `Basic_Types.Byte_Array` – there is no ‘Address reinterpretation’ – so the dynamic-argument machinery is type-safe by construction; a resolver cannot overlay an arbitrary record onto raw memory, only deserialize a byte array through the input type’s generated deserializer.

8 Sleep Steps

A sequence step may be a sleep instead of a command: `sleep_ms:` pauses the executing frame without issuing any bus traffic. Sleep is not a bus command; it is interpreted intrinsically by the sequencer frame state machine. The duration takes two forms. A *static* sleep (`sleep_ms: 3000`) is a plain integer number of milliseconds fixed at build time; the model bounds it to $0 \dots 2^{31} - 1$ so it always fits an `Ada.Real_Time.Time_Span` by construction, leaving wake-time overflow of `Sys_Time` arithmetic as its only run-time failure. A *dynamic* sleep (`sleep_ms: Arg.Settle_Time`) instead references the sequence’s per-call argument with the same `Arg[.field]` syntax as dynamic command arguments; the referenced field must be a `Packed_Natural.T`, resolved at execution time through the same validating Resolver machinery, so the duration is range-checked as part of argument validation (a failure aborts the sequence with an `Invalid_Dynamic_Sleep_Argument` event) and always fits a `Time_Span`. A sleep whose wake-time computation overflows ends the sequence with a `Sequence_Out_Of_Range_Sleep` event and a `Failure` status to any pending deferred reply – continuing would leave the frame parked on a stale wake time.

9 Summary Packet Type

The sequencer emits a periodic `Summary_Packet` containing one `Sequence_Frame_Summary` record per sequence frame, in frame order. Its layout therefore depends on the suite’s `num_concurrent_sequences`, so the generator also autocodes the packet’s ground/documentation record type, `<suite>_summary_record.record.yaml`, alongside the suite’s other outputs. The flight software never imports this type – the component fills the packet buffer directly from its frame array – but ground tools and assembly documentation need it. At assembly load time the sequencer’s packets model resolves an instance’s `Summary_Packet` type to the summary record of the suite named by the instance’s `Config init` parameter. Because the frame pool and the packet layout both come from the suite model’s single `num_concurrent_sequences` value, they can never disagree; the model also rejects at build time any suite whose frame summaries would not fit in one packet buffer (the same bound the `Num_Concurrent_Sequences_Type` subtype enforces at `Init`). The example input above generates the following record model:

```
1  ---
2  description: Autocoded summary packet type for the
   ↳ Test_Assembly_Command_Sequences_Example_Sequences command sequences suite.
   ↳ Contains one Sequence_Frame_Summary record per sequence frame (2 total, from
   ↳ the suite's num_concurrent_sequences), in frame order.
3  fields:
4    - name: Frame_0_Summary
5      description: "A summary of the state of sequence frame 0."
6      type: Sequence_Frame_Summary.T
7    - name: Frame_1_Summary
8      description: "A summary of the state of sequence frame 1."
9      type: Sequence_Frame_Summary.T
10
```